Bucles e iteraciones en Python

Edison Achalma
Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

Este abstract será actualizado una vez que se complete el contenido final del artículo.

Tabla de contenidos

Introduction **3**

1 Actualización de variables **3**
1.1 Operadores de actualización . . . 3
1.2 Aplicaciones económicas . . . 3

2 Bucles definidos con for **4**
2.1 Función range() . . . 4
2.2 Aplicaciones económicas básicas . . . 5

3 Iteración sobre colecciones **6**
3.1 Iterar sobre listas . . . 6
3.2 Iterar sobre tuplas . . . 6
3.3 Iterar sobre diccionarios . . . 6
3.4 Iterar sobre strings . . . 7

4 List comprehension **8**
4.1 Sintaxis básica . . . 8
4.2 List comprehension con condicionales . . . 8
4.3 Aplicaciones económicas . . . 9
4.4 Comprehensions anidadas . . . 9
4.5 Dictionary comprehension . . . 10

Edison Achalma https://orcid.org/0000-0001-6996-3364

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; https://credit.niso.org/) de la siguiente manera: Edison Achalma: conceptualización, metodología, análisis formal, investigación, recursos, curación de datos, redacción, visualización, supervisión, administración del proyecto

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga, Portal Independencia N° 57, Ayacucho, AYA 5001, Perú, Email: elmer.achalma.09@unsch.edu.pe

5 Iteración con condicionales **10**
5.1 Filtrado durante iteración . . . 10
5.2 Transformación condicional . . . 11
5.3 Múltiples condiciones . . . 12

6 Iteración múltiple y paralela **12**
6.1 Función zip() . . . 12
6.2 Aplicaciones económicas . . . 13
6.3 Crear diccionarios con zip() . . . 13

7 Iteraciones anidadas **13**
7.1 Matrices y tablas . . . 14
7.2 Aplicaciones económicas . . . 14
7.3 Búsqueda en estructuras anidadas . . . 15

8 Bucles while **16**
8.1 Sintaxis básica . . . 16
8.2 Aplicaciones económicas . . . 16
8.3 Bucles infinitos y salida . . . 18

9 Control de flujo: break y continue **18**
9.1 break: terminar bucle . . . 18
9.2 continue: saltar iteración . . . 18
9.3 Aplicaciones combinadas . . . 19

10 Aplicaciones económicas **19**
10.1 Aplicación 1: Simulador de portafolio de inversión . . . 19
10.2 Aplicación 2: Análisis de sensibilidad de precios . . . 21

11 Ejercicios prácticos **23**
11.1 Ejercicio 1: Calculadora de depreciación . . . 23
11.2 Ejercicio 2: Simulador de política de inventarios . . . 24

12 Conlusión **26**

13 Próximos pasos **27**

14 Recursos adicionales **27**

15 Publicaciones Similares **27**

Bucles e iteraciones en Python

En esta quinta guía exploraremos las iteraciones, una de las herramientas más poderosas de la programación. Las iteraciones nos permiten procesar grandes cantidades de datos económicos de manera automatizada y eficiente.

1 Actualización de variables

La actualización de variables es fundamental en iteraciones. Consiste en modificar el valor de una variable usando su valor previo.

1.1 Operadores de actualización

```
# Inicializar contador
contador = 0
print(f"Valor inicial: {contador}")

# Incrementar
contador = contador + 1
print(f"Después de +1: {contador}")

# Forma abreviada (más común)
contador += 1
print(f"Después de +=1: {contador}")

# Decrementar
contador -= 1
print(f"Después de -=1: {contador}")

# Multiplicar
contador *= 2
print(f"Después de *=2: {contador}")

# Dividir
contador /= 2
print(f"Después de /=2: {contador}")

# Potencia
contador **= 2
print(f"Después de **=2: {contador}")
```

1.2 Aplicaciones económicas

```
# Ejemplo 1: Acumular ventas diarias
ventas_total = 0
ventas_diarias = [1200, 1350, 980, 1500, 1100]

for venta in ventas_diarias:
```

```
    ventas_total += venta

print(f"Ventas totales: S/ {ventas_total:,.2f}")

# Ejemplo 2: Calcular interés compuesto
capital = 10000
tasa_anual = 0.08
anos = 5

for ano in range(anos):
    interes = capital * tasa_anual
    capital += interes
    print(f"Año {ano + 1}: S/ {capital:,.2f}")

print(f"\nCapital final: S/ {capital:,.2f}")

# Ejemplo 3: Contador de transacciones por tipo
num_compras = 0
num_ventas = 0
num_transferencias = 0

transacciones = ['compra', 'venta', 'compra', 'transferencia', 'venta', 'compra']

for tipo in transacciones:
    if tipo == 'compra':
        num_compras += 1
    elif tipo == 'venta':
        num_ventas += 1
    elif tipo == 'transferencia':
        num_transferencias += 1

print(f"Compras: {num_compras}")
print(f"Ventas: {num_ventas}")
print(f"Transferencias: {num_transferencias}")
```

2 Bucles definidos con for

Los bucles `for` permiten iterar sobre una secuencia de elementos un número determinado de veces.

2.1 Función range()

```
# range(stop): de 0 hasta stop-1
print("range(5):")
for i in range(5):
    print(i)
```

```python
# range(start, stop): de start hasta stop-1
print("\nrange(2, 7):")
for i in range(2, 7):
    print(i)

# range(start, stop, step): de start hasta stop-1 con paso
print("\nrange(0, 10, 2):")
for i in range(0, 10, 2):
    print(i)

# Paso negativo
print("\nrange(10, 0, -1):")
for i in range(10, 0, -1):
    print(i)
```

2.2 Aplicaciones económicas básicas

```python
# Ejemplo 1: Generar serie de años
print("Proyección económica 2024-2033:")
for ano in range(2024, 2034):
    print(f"Año {ano}")

# Ejemplo 2: Calcular tabla de amortización simplificada
print("\nTabla de amortización (primeros 5 años):")
prestamo = 100000
tasa_mensual = 0.01
cuota = 2000

for mes in range(1, 61):
    interes = prestamo * tasa_mensual
    amortizacion = cuota - interes
    prestamo -= amortizacion

    if mes % 12 == 0:
        print(f"Año {mes//12}: Saldo restante S/ {prestamo:,.2f}")

# Ejemplo 3: Simulación de inversión mensual
print("\nSimulación de ahorro mensual:")
ahorro = 0
aporte_mensual = 500

for mes in range(1, 13):
    ahorro += aporte_mensual
    print(f"Mes {mes:2d}: S/ {ahorro:,.2f}")

print(f"\nAhorro anual: S/ {ahorro:,.2f}")
```

3 Iteración sobre colecciones

Los bucles `for` pueden iterar directamente sobre listas, tuplas, diccionarios y otros objetos iterables.

3.1 Iterar sobre listas

```
# Lista de sectores económicos
sectores = ['Agricultura', 'Minería', 'Manufactura', 'Servicios', 'Construcción']

print("Sectores económicos:")
for sector in sectores:
    print(f"  - {sector}")

# Con índice usando enumerate()
print("\nSectores con índice:")
for i, sector in enumerate(sectores):
    print(f"{i + 1}. {sector}")

# Desde índice específico
print("\nDesde el segundo sector:")
for i, sector in enumerate(sectores, start=1):
    print(f"{i}. {sector}")
```

3.2 Iterar sobre tuplas

```
# Tupla de indicadores
indicadores = ('PIB', 'Inflación', 'Desempleo', 'Tipo de cambio')

print("Indicadores macroeconómicos:")
for indicador in indicadores:
    print(f"  {indicador}")
```

3.3 Iterar sobre diccionarios

```
# Diccionario de datos económicos
datos_peru = {
    'pib': 242632,
    'poblacion': 33715471,
    'inflacion': 3.5,
    'desempleo': 7.2
}

# Iterar sobre claves
print("Claves:")
for clave in datos_peru.keys():
    print(f"  {clave}")
```

```python
# Iterar sobre valores
print("\nValores:")
for valor in datos_peru.values():
    print(f"  {valor}")

# Iterar sobre items (clave-valor)
print("\nIndicadores del Perú:")
for indicador, valor in datos_peru.items():
    print(f"  {indicador}: {valor}")

# Formato mejorado
print("\nReporte económico:")
for indicador, valor in datos_peru.items():
    if indicador == 'pib':
        print(f"  PIB: USD {valor:,} millones")
    elif indicador == 'poblacion':
        print(f"  Población: {valor:,} habitantes")
    elif indicador in ['inflacion', 'desempleo']:
        print(f"  {indicador.capitalize()}: {valor}%")
```

3.4 Iterar sobre strings

```python
# Strings son iterables
codigo = "CIIU-2023"

print("Caracteres del código:")
for caracter in codigo:
    print(caracter)

# Aplicación: validar formato
letras = 0
numeros = 0
otros = 0

for caracter in codigo:
    if caracter.isalpha():
        letras += 1
    elif caracter.isdigit():
        numeros += 1
    else:
        otros += 1

print(f"\nAnálisis del código '{codigo}':")
print(f"  Letras: {letras}")
print(f"  Números: {numeros}")
print(f"  Otros: {otros}")
```

4 List comprehension

Las list comprehensions son una forma concisa y elegante de crear listas. Son más eficientes y legibles que los bucles tradicionales.

4.1 Sintaxis básica

```
# Forma tradicional con for
cuadrados_for = []
for i in range(10):
    cuadrados_for.append(i ** 2)

print(f"Con for: {cuadrados_for}")

# Con list comprehension
cuadrados_lc = [i ** 2 for i in range(10)]
print(f"Con LC:  {cuadrados_lc}")

# Más ejemplos
# Generar lista de años
anos = [2020 + i for i in range(10)]
print(f"\nAños: {anos}")

# Convertir a strings
anos_str = [str(ano) for ano in anos]
print(f"Años (str): {anos_str}")

# Operaciones con elementos
precios = [100, 120, 150, 80, 95]
precios_con_igv = [precio * 1.18 for precio in precios]
print(f"\nPrecios sin IGV: {precios}")
print(f"Precios con IGV: {precios_con_igv}")
```

4.2 List comprehension con condicionales

```
# Sintaxis: [expresion for item in iterable if condicion]

# Filtrar números pares
numeros = list(range(20))
pares = [n for n in numeros if n % 2 == 0]
print(f"Pares: {pares}")

# Filtrar impares
impares = [n for n in numeros if n % 2 != 0]
print(f"Impares: {impares}")

# Múltiplos de 3
```

```
multiplos_3 = [n for n in numeros if n % 3 == 0]
print(f"Múltiplos de 3: {multiplos_3}")

# Múltiplos de 2 o 3
multiplos_2_o_3 = [n for n in numeros if n % 2 == 0 or n % 3 == 0]
print(f"Múltiplos de 2 o 3: {multiplos_2_o_3}")
```

4.3 Aplicaciones económicas

```
# Ejemplo 1: Filtrar datos económicos
pib_regional = [100000, 50000, 75000, 30000, 45000, 85000, 60000]

# Regiones con PIB > 50000
grandes_economias = [pib for pib in pib_regional if pib > 50000]
print(f"Grandes economías: {grandes_economias}")

# Ejemplo 2: Calcular variaciones porcentuales
precios_2022 = [100, 105, 110, 108, 112]
precios_2023 = [105, 110, 115, 112, 118]

variaciones = [((p2 - p1) / p1) * 100
               for p1, p2 in zip(precios_2022, precios_2023)]

print(f"\nVariaciones de precios:")
for i, var in enumerate(variaciones, 1):
    print(f"  Producto {i}: {var:+.2f}%")

# Ejemplo 3: Clasificar empresas por tamaño
ventas = [500000, 2000000, 15000000, 800000, 25000000, 1200000]

clasificacion = ['Pequeña' if v < 1000000
                 else 'Mediana' if v < 10000000
                 else 'Grande'
                 for v in ventas]

print(f"\nClasificación de empresas:")
for i, (venta, cat) in enumerate(zip(ventas, clasificacion), 1):
    print(f"  Empresa {i}: S/ {venta:,} - {cat}")
```

4.4 Comprehensions anidadas

```
# Crear matriz
matriz = [[i * j for j in range(5)] for i in range(5)]

print("Matriz de multiplicación:")
for fila in matriz:
```

```
    print(fila)

# Aplanar lista de listas
lista_anidada = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
lista_plana = [num for sublista in lista_anidada for num in sublista]
print(f"\nLista anidada: {lista_anidada}")
print(f"Lista plana: {lista_plana}")
```

4.5 Dictionary comprehension

```
# Crear diccionarios con comprehension
# Sintaxis: {clave: valor for item in iterable}

# Ejemplo 1: Cuadrados
cuadrados_dict = {x: x**2 for x in range(6)}
print(f"Cuadrados: {cuadrados_dict}")

# Ejemplo 2: Mapeo de códigos
productos = ['A', 'B', 'C', 'D', 'E']
precios = [10, 15, 20, 25, 30]

catalogo = {prod: precio for prod, precio in zip(productos, precios)}
print(f"\nCatálogo: {catalogo}")

# Ejemplo 3: Filtrar diccionario
datos_economicos = {
    'pib': 242632,
    'poblacion': 33715471,
    'inflacion': 3.5,
    'desempleo': 7.2,
    'reservas': 74500
}

# Solo valores mayores a 100
grandes_valores = {k: v for k, v in datos_economicos.items() if v > 100}
print(f"\nGrandes valores: {grandes_valores}")
```

5 Iteración con condicionales

Combinar iteraciones con condicionales permite procesar datos de manera selectiva.

5.1 Filtrado durante iteración

```
# Datos de transacciones
transacciones = [
    {'monto': 1500, 'tipo': 'ingreso'},
    {'monto': 800, 'tipo': 'gasto'},
```

```
    {'monto': 2000, 'tipo': 'ingreso'},
    {'monto': 500, 'tipo': 'gasto'},
    {'monto': 1200, 'tipo': 'ingreso'},
]

# Separar por tipo
ingresos = []
gastos = []

for trans in transacciones:
    if trans['tipo'] == 'ingreso':
        ingresos.append(trans['monto'])
    else:
        gastos.append(trans['monto'])

print(f"Ingresos: S/ {sum(ingresos):,.2f}")
print(f"Gastos: S/ {sum(gastos):,.2f}")
print(f"Balance: S/ {sum(ingresos) - sum(gastos):,.2f}")

# Con list comprehension
ingresos_lc = [t['monto'] for t in transacciones if t['tipo'] == 'ingreso']
gastos_lc = [t['monto'] for t in transacciones if t['tipo'] == 'gasto']

print(f"\nCon LC:")
print(f"Ingresos: S/ {sum(ingresos_lc):,.2f}")
print(f"Gastos: S/ {sum(gastos_lc):,.2f}")
```

5.2 Transformación condicional

```
# Clasificar trabajadores por género según nombre
nombres = ['María', 'Juan', 'Carmen', 'Pedro', 'Rosa', 'Carlos', 'Ana', 'Miguel']

trabajadores = []
for nombre in nombres:
    if nombre[-1] == 'a':
        trabajadores.append(f"Trabajadora: {nombre}")
    else:
        trabajadores.append(f"Trabajador: {nombre}")

print("Lista de trabajadores:")
for trabajador in trabajadores:
    print(f"  {trabajador}")

# Con list comprehension
trabajadores_lc = [f"Trabajador{'a' if nombre[-1] == 'a' else ''}: {nombre}"
                   for nombre in nombres]
```

```
print("\nCon LC:")
for trabajador in trabajadores_lc:
    print(f"  {trabajador}")
```

5.3 Múltiples condiciones

```
# Clasificar números por propiedades
numeros = list(range(1, 21))

pares = []
impares = []
multiplos_3 = []
multiplos_5 = []

for n in numeros:
    if n % 2 == 0:
        pares.append(n)
    else:
        impares.append(n)

    if n % 3 == 0:
        multiplos_3.append(n)

    if n % 5 == 0:
        multiplos_5.append(n)

print(f"Pares: {pares}")
print(f"Impares: {impares}")
print(f"Múltiplos de 3: {multiplos_3}")
print(f"Múltiplos de 5: {multiplos_5}")
```

6 Iteración múltiple y paralela

La función `zip()` permite iterar sobre múltiples secuencias en paralelo.

6.1 Función zip()

```
# Listas paralelas
paises = ['Perú', 'Chile', 'Argentina', 'Colombia']
pib = [242632, 301000, 487000, 314000]
poblacion = [33715471, 19493184, 45810000, 51516562]

# Iterar en paralelo
print("Datos económicos de países:")
for pais, pib_val, pob in zip(paises, pib, poblacion):
    pib_pc = (pib_val / pob) * 1000000
    print(f"{pais:12} - PIB: USD {pib_val:,} M - Pob: {pob:,} - PIB/c: USD {pib_pc:,.
```

6.2 Aplicaciones económicas

```
# Ejemplo 1: Análisis de series de tiempo
meses = ['Ene', 'Feb', 'Mar', 'Abr', 'May', 'Jun']
ventas = [45000, 52000, 48000, 55000, 60000, 58000]
costos = [30000, 35000, 32000, 37000, 40000, 39000]

print("Análisis mensual de rentabilidad:")
print(f"{'Mes':<5} {'Ventas':>10} {'Costos':>10} {'Utilidad':>10} {'Margen':>8}")
print("-" * 50)

for mes, venta, costo in zip(meses, ventas, costos):
    utilidad = venta - costo
    margen = (utilidad / venta) * 100
    print(f"{mes:<5} {venta:>10,} {costo:>10,} {utilidad:>10,} {margen:>7.1f}%")

# Ejemplo 2: Comparar presupuesto vs real
categorias = ['Salarios', 'Alquiler', 'Servicios', 'Marketing', 'Otros']
presupuestado = [80000, 15000, 8000, 12000, 5000]
gastado = [85000, 15000, 7500, 14000, 4800]

print("\n\nAnálisis de ejecución presupuestaria:")
print(f"{'Categoría':<12} {'Presup.':>10} {'Gastado':>10} {'Dif.':>10} {'%':>6}")
print("-" * 55)

for cat, pres, gasto in zip(categorias, presupuestado, gastado):
    diferencia = gasto - pres
    porcentaje = (gasto / pres) * 100
    signo = "+" if diferencia > 0 else ""
    print(f"{cat:<12} {pres:>10,} {gasto:>10,} {signo}{diferencia:>9,} {porcentaje:>5
```

6.3 Crear diccionarios con zip()

```
# Crear diccionario desde dos listas
productos = ['Laptop', 'Mouse', 'Teclado', 'Monitor']
precios = [2500, 25, 80, 800]

catalogo = dict(zip(productos, precios))
print(f"Catálogo: {catalogo}")

# Acceder
print(f"\nPrecio de Laptop: S/ {catalogo['Laptop']:,.2f}")
```

7 Iteraciones anidadas

Las iteraciones anidadas permiten trabajar con estructuras de datos multidimensionales.

7.1 Matrices y tablas

```
# Crear matriz de ceros
filas = 3
columnas = 4

matriz = []
for i in range(filas):
    fila = []
    for j in range(columnas):
        fila.append(0)
    matriz.append(fila)

print("Matriz de ceros:")
for fila in matriz:
    print(fila)

# Con list comprehension
matriz_lc = [[0 for j in range(columnas)] for i in range(filas)]

print("\nCon LC:")
for fila in matriz_lc:
    print(fila)
```

7.2 Aplicaciones económicas

```
# Ejemplo 1: Tabla de multiplicación (escalas de producción)
print("Tabla de costos de producción:")
print("     ", end="")
for cantidad in range(1, 6):
    print(f"{cantidad*10:>6}", end="")
print()

for precio in range(1, 6):
    print(f"{precio*100:>5}", end=" ")
    for cantidad in range(1, 6):
        costo = precio * 100 * cantidad * 10
        print(f"{costo:>6}", end="")
    print()

# Ejemplo 2: Análisis regional-sectorial
regiones = ['Lima', 'Arequipa', 'Cusco']
sectores = ['Agricultura', 'Minería', 'Manufactura']

# Datos de PIB por región y sector (millones)
pib_regional_sectorial = [
    [5000, 15000, 30000],  # Lima
```

```
    [8000, 12000, 5000],   # Arequipa
    [6000, 8000, 3000]     # Cusco
]

print("\n\nPIB por región y sector (millones USD):")
print(f"{'Región':<12}", end="")
for sector in sectores:
    print(f"{sector:>15}", end="")
print(f"{'Total':>15}")
print("-" * 70)

for i, region in enumerate(regiones):
    print(f"{region:<12}", end="")
    total_region = 0
    for j, sector in enumerate(sectores):
        valor = pib_regional_sectorial[i][j]
        print(f"{valor:>15,}", end="")
        total_region += valor
    print(f"{total_region:>15,}")

# Totales por sector
print(f"{'Total':<12}", end="")
total_general = 0
for j in range(len(sectores)):
    total_sector = sum(pib_regional_sectorial[i][j] for i in range(len(regiones)))
    print(f"{total_sector:>15,}", end="")
    total_general += total_sector
print(f"{total_general:>15,}")
```

7.3 Búsqueda en estructuras anidadas

```
# Buscar en matriz
matriz_datos = [
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
]

valor_buscar = 50
encontrado = False

for i, fila in enumerate(matriz_datos):
    for j, valor in enumerate(fila):
        if valor == valor_buscar:
            print(f"Valor {valor_buscar} encontrado en posición [{i}][{j}]")
            encontrado = True
            break
```

```
    if encontrado:
        break

if not encontrado:
    print(f"Valor {valor_buscar} no encontrado")
```

8 Bucles while

Los bucles `while` se ejecutan mientras una condición sea verdadera. Son útiles cuando no sabemos de antemano cuántas iteraciones necesitamos.

8.1 Sintaxis básica

```
# Contador simple
contador = 0
while contador < 5:
    print(f"Iteración {contador}")
    contador += 1

print("Fin del bucle")
```

8.2 Aplicaciones económicas

```
# Ejemplo 1: Simular inversión hasta alcanzar meta
capital = 10000
meta = 20000
tasa_anual = 0.08
anos = 0

print(f"Capital inicial: S/ {capital:,.2f}")
print(f"Meta: S/ {meta:,.2f}")
print(f"Tasa: {tasa_anual*100}%\n")

while capital < meta:
    anos += 1
    interes = capital * tasa_anual
    capital += interes
    print(f"Año {anos}: S/ {capital:,.2f}")

print(f"\nMeta alcanzada en {anos} años")

# Ejemplo 2: Búsqueda de punto de equilibrio
precio = 100
costo_variable = 40
costos_fijos = 50000
cantidad = 0
ingresos = 0
```

```
costos_totales = costos_fijos

print(f"\nBúsqueda de punto de equilibrio:")
print(f"Precio: S/ {precio}")
print(f"Costo variable: S/ {costo_variable}")
print(f"Costos fijos: S/ {costos_fijos:,}\n")

while ingresos < costos_totales:
    cantidad += 100
    ingresos = precio * cantidad
    costos_totales = costos_fijos + (costo_variable * cantidad)

print(f"Punto de equilibrio: {cantidad:,} unidades")
print(f"Ingresos: S/ {ingresos:,}")
print(f"Costos totales: S/ {costos_totales:,}")

# Ejemplo 3: Pago de deuda con cuotas fijas
deuda = 50000
tasa_mensual = 0.015
cuota = 2000
mes = 0
interes_total = 0

print(f"\n\nAmortización de deuda:")
print(f"Deuda inicial: S/ {deuda:,.2f}")
print(f"Cuota mensual: S/ {cuota:,.2f}")
print(f"Tasa mensual: {tasa_mensual*100}%\n")

while deuda > 0:
    mes += 1
    interes = deuda * tasa_mensual
    interes_total += interes
    amortizacion = cuota - interes

    if amortizacion > deuda:
        amortizacion = deuda
        cuota = deuda + interes

    deuda -= amortizacion

    if mes % 12 == 0:
        print(f"Mes {mes:2d}: Saldo S/ {deuda:,.2f}")

print(f"\nDeuda pagada en {mes} meses ({mes/12:.1f} años)")
print(f"Interés total pagado: S/ {interes_total:,.2f}")
```

8.3 Bucles infinitos y salida

```
# Bucle con condición de salida
print("Simulación de proceso hasta condición:")
iteracion = 0
valor = 100

while True:
    iteracion += 1
    valor *= 0.9  # Decrece 10% cada iteración

    print(f"Iteración {iteracion}: {valor:.2f}")

    if valor < 10:
        print("Valor crítico alcanzado")
        break

    if iteracion > 100:
        print("Límite de iteraciones alcanzado")
        break
```

9 Control de flujo: break y continue

`break` y `continue` permiten controlar el flujo de las iteraciones.

9.1 break: terminar bucle

```
# Buscar primer valor que cumple condición
precios = [100, 105, 110, 95, 120, 130, 90]
umbral = 115

print(f"Buscando primer precio mayor a {umbral}:")
for i, precio in enumerate(precios):
    print(f"Revisando posición {i}: S/ {precio}")
    if precio > umbral:
        print(f"Encontrado en posición {i}: S/ {precio}")
        break
else:
    print("No se encontró ningún precio que supere el umbral")
```

9.2 continue: saltar iteración

```
# Procesar solo valores válidos
datos = [100, -5, 200, 0, 150, -10, 300, 250]

print("Procesando solo valores positivos:")
suma = 0
```

```python
for valor in datos:
    if valor <= 0:
        print(f"Saltando valor inválido: {valor}")
        continue

    suma += valor
    print(f"Procesando: {valor}, suma acumulada: {suma}")

print(f"\nSuma total: {suma}")
```

9.3 Aplicaciones combinadas

```python
# Validación de datos con límite de intentos
intentos_maximos = 3
intentos = 0

while intentos < intentos_maximos:
    try:
        monto = float(input("Ingrese monto (positivo): "))

        if monto < 0:
            print("Error: el monto debe ser positivo")
            intentos += 1
            continue

        if monto > 100000:
            print("Advertencia: monto muy alto")
            confirmar = input("¿Desea continuar? (s/n): ")
            if confirmar.lower() != 's':
                continue

        print(f"Monto válido: S/ {monto:,.2f}")
        break

    except ValueError:
        print("Error: ingrese un número válido")
        intentos += 1

if intentos >= intentos_maximos:
    print("\nNúmero máximo de intentos alcanzado")
```

10 Aplicaciones económicas

10.1 Aplicación 1: Simulador de portafolio de inversión

```python
def simular_portafolio(inversion_inicial, anos, distribucion, rendimientos):
    """
    Simula el rendimiento de un portafolio diversificado

    Parámetros:
    - inversion_inicial: capital inicial
    - anos: número de años a simular
    - distribucion: dict con % por activo
    - rendimientos: dict con rendimientos anuales por activo
    """

    print("SIMULACIÓN DE PORTAFOLIO DE INVERSIÓN")
    print("=" * 70)

    # Calcular inversión por activo
    inversiones = {}
    print(f"\nInversión inicial: S/ {inversion_inicial:,.2f}")
    print(f"\nDistribución del portafolio:")

    for activo, porcentaje in distribucion.items():
        monto = inversion_inicial * porcentaje
        inversiones[activo] = monto
        print(f"  {activo}: {porcentaje*100:.0f}% - S/ {monto:,.2f}")

    # Simular cada año
    print(f"\n{'='*70}")
    print("Evolución anual del portafolio:")
    print(f"{'Año':<6} ", end="")
    for activo in inversiones.keys():
        print(f"{activo:>12}", end="")
    print(f"{'Total':>12}")
    print("-" * 70)

    for ano in range(1, anos + 1):
        total_ano = 0
        print(f"{ano:<6} ", end="")

        for activo in inversiones.keys():
            rendimiento = rendimientos[activo]
            inversiones[activo] *= (1 + rendimiento)
            print(f"{inversiones[activo]:>12,.0f}", end="")
            total_ano += inversiones[activo]

        print(f"{total_ano:>12,.0f}")

    # Resultados finales
    print(f"\n{'='*70}")
```

```
    print("RESULTADOS FINALES:")

    total_final = sum(inversiones.values())
    ganancia = total_final - inversion_inicial
    rendimiento_total = (ganancia / inversion_inicial) * 100
    rendimiento_anual = ((total_final / inversion_inicial) ** (1/anos) - 1) * 100

    print(f"\nInversión inicial: S/ {inversion_inicial:,.2f}")
    print(f"Valor final: S/ {total_final:,.2f}")
    print(f"Ganancia: S/ {ganancia:,.2f}")
    print(f"Rendimiento total: {rendimiento_total:.2f}%")
    print(f"Rendimiento anual promedio: {rendimiento_anual:.2f}%")

    print(f"\nDistribución final:")
    for activo, monto in inversiones.items():
        porcentaje = (monto / total_final) * 100
        print(f"  {activo}: S/ {monto:,.2f} ({porcentaje:.1f}%)")

    return inversiones

# Ejemplo de uso
distribucion = {
    'Acciones': 0.40,
    'Bonos': 0.30,
    'Inmuebles': 0.20,
    'Efectivo': 0.10
}

rendimientos = {
    'Acciones': 0.12,
    'Bonos': 0.06,
    'Inmuebles': 0.08,
    'Efectivo': 0.02
}

resultado = simular_portafolio(
    inversion_inicial=100000,
    anos=10,
    distribucion=distribucion,
    rendimientos=rendimientos
)
```

10.2 Aplicación 2: Análisis de sensibilidad de precios

```
def analisis_sensibilidad_demanda(precio_base, elasticidad, cantidad_base,
                                  variaciones_precio):
    """
```

```
    Analiza cómo cambios en el precio afectan la demanda
    """

    print("ANÁLISIS DE SENSIBILIDAD DE LA DEMANDA")
    print("=" * 70)

    print(f"\nParámetros base:")
    print(f"  Precio base: S/ {precio_base:.2f}")
    print(f"  Cantidad base: {cantidad_base:,} unidades")
    print(f"  Elasticidad precio: {elasticidad:.2f}")

    print(f"\n{'='*70}")
    print("Escenarios de precios:")
    print(f"{'Precio':>10} {'Var %':>8} {'Cantidad':>12} {'Var %':>8} {'Ingreso':>12}")
    print("-" * 70)

    resultados = []

    for var_porcentaje in variaciones_precio:
        # Calcular nuevo precio
        nuevo_precio = precio_base * (1 + var_porcentaje/100)

        # Calcular cambio porcentual en cantidad (elasticidad)
        var_cantidad_porcentaje = -elasticidad * var_porcentaje
        nueva_cantidad = cantidad_base * (1 + var_cantidad_porcentaje/100)

        # Calcular ingreso
        ingreso = nuevo_precio * nueva_cantidad
        ingreso_base = precio_base * cantidad_base
        var_ingreso = ((ingreso - ingreso_base) / ingreso_base) * 100

        print(f"{nuevo_precio:>10.2f} {var_porcentaje:>7.1f}% "
              f"{nueva_cantidad:>12,.0f} {var_cantidad_porcentaje:>7.1f}% "
              f"{ingreso:>12,.0f}")

        resultados.append({
            'precio': nuevo_precio,
            'cantidad': nueva_cantidad,
            'ingreso': ingreso
        })

    # Encontrar precio óptimo
    precio_optimo_idx = max(range(len(resultados)),
                            key=lambda i: resultados[i]['ingreso'])

    print(f"\n{'='*70}")
    print("PRECIO ÓPTIMO:")
```

```
    optimo = resultados[precio_optimo_idx]
    print(f"  Precio: S/ {optimo['precio']:.2f}")
    print(f"  Cantidad: {optimo['cantidad']:,.0f} unidades")
    print(f"  Ingreso máximo: S/ {optimo['ingreso']:,.2f}")

    return resultados

# Ejemplo de uso
variaciones = [-20, -15, -10, -5, 0, 5, 10, 15, 20]
resultados = analisis_sensibilidad_demanda(
    precio_base=100,
    elasticidad=1.5,
    cantidad_base=1000,
    variaciones_precio=variaciones
)
```

11 Ejercicios prácticos

11.1 Ejercicio 1: Calculadora de depreciación

```
def calcular_depreciacion(valor_inicial, vida_util, metodo='lineal'):
    """
    Calcula la depreciación de un activo

    Métodos:
    - 'lineal': Depreciación constante
    - 'doble_saldo': Saldo decreciente acelerado
    """

    print(f"CÁLCULO DE DEPRECIACIÓN - Método {metodo.upper()}")
    print("=" * 60)

    print(f"\nValor inicial: S/ {valor_inicial:,.2f}")
    print(f"Vida útil: {vida_util} años")

    if metodo == 'lineal':
        # Depreciación lineal
        depreciacion_anual = valor_inicial / vida_util
        valor_libro = valor_inicial

        print(f"Depreciación anual: S/ {depreciacion_anual:,.2f}")
        print(f"\n{'Año':<6} {'Deprec.':<12} {'Deprec. Acum.':<15} {'Valor Libro':<12
        print("-" * 60)

        depreciacion_acumulada = 0

        for ano in range(1, vida_util + 1):
```

```
            depreciacion_acumulada += depreciacion_anual
            valor_libro -= depreciacion_anual

            print(f"{ano:<6} {depreciacion_anual:>11,.2f} "
                  f"{depreciacion_acumulada:>14,.2f} {valor_libro:>11,.2f}")

    elif metodo == 'doble_saldo':
        # Depreciación por doble saldo decreciente
        tasa = 2 / vida_util
        valor_libro = valor_inicial
        depreciacion_acumulada = 0

        print(f"Tasa anual: {tasa*100:.2f}%")
        print(f"\n{'Año':<6} {'Valor Inicio':<13} {'Deprec.':<12} "
              f"{'Deprec. Acum.':<15} {'Valor Final':<12}")
        print("-" * 70)

        for ano in range(1, vida_util + 1):
            valor_inicio = valor_libro
            depreciacion = valor_libro * tasa

            # En el último año, depreciar solo hasta valor residual cero
            if ano == vida_util:
                depreciacion = valor_libro

            depreciacion_acumulada += depreciacion
            valor_libro -= depreciacion

            print(f"{ano:<6} {valor_inicio:>12,.2f} {depreciacion:>11,.2f} "
                  f"{depreciacion_acumulada:>14,.2f} {valor_libro:>11,.2f}")

    print("=" * 60)

# Ejemplos
print("DEPRECIACIÓN LINEAL")
calcular_depreciacion(valor_inicial=50000, vida_util=5, metodo='lineal')

print("\n\n")

print("DEPRECIACIÓN POR DOBLE SALDO DECRECIENTE")
calcular_depreciacion(valor_inicial=50000, vida_util=5, metodo='doble_saldo')
```

11.2 Ejercicio 2: Simulador de política de inventarios

```
def simular_politica_inventario(demanda_diaria, dias_simulacion,
                                punto_reorden, cantidad_pedido,
                                tiempo_entrega, inventario_inicial):
```

```
    """
    Simula una política de inventarios (Q, R)
    """

    import random

    print("SIMULACIÓN DE POLÍTICA DE INVENTARIOS")
    print("=" * 70)

    print(f"\nParámetros:")
    print(f"  Demanda diaria promedio: {demanda_diaria} unidades")
    print(f"  Inventario inicial: {inventario_inicial} unidades")
    print(f"  Punto de reorden: {punto_reorden} unidades")
    print(f"  Cantidad de pedido: {cantidad_pedido} unidades")
    print(f"  Tiempo de entrega: {tiempo_entrega} días")

    # Variables de simulación
    inventario = inventario_inicial
    pedido_pendiente = False
    dias_hasta_entrega = 0
    total_vendido = 0
    dias_sin_stock = 0
    num_pedidos = 0
    ventas_perdidas = 0

    print(f"\n{'Día':<5} {'Demanda':<8} {'Ventas':<8} {'Inventario':<10} {'Pedido':<8
    print("-" * 50)

    for dia in range(1, dias_simulacion + 1):
        # Generar demanda (distribución normal)
        demanda = max(0, int(random.normalvariate(demanda_diaria, demanda_diaria * 0.

        # Verificar si llega pedido
        if pedido_pendiente:
            dias_hasta_entrega -= 1
            if dias_hasta_entrega == 0:
                inventario += cantidad_pedido
                pedido_pendiente = False

        # Procesar venta
        if inventario >= demanda:
            ventas = demanda
            inventario -= demanda
            total_vendido += ventas
        else:
            ventas = inventario
            ventas_perdidas += (demanda - inventario)
```

```
            total_vendido += ventas
            inventario = 0
            dias_sin_stock += 1

        # Verificar punto de reorden
        estado_pedido = ""
        if inventario <= punto_reorden and not pedido_pendiente:
            pedido_pendiente = True
            dias_hasta_entrega = tiempo_entrega
            num_pedidos += 1
            estado_pedido = "PEDIDO"

        # Mostrar cada 5 días
        if dia % 5 == 0 or dia == 1:
            print(f"{dia:<5} {demanda:<8} {ventas:<8} {inventario:<10} {estado_pedido

    # Estadísticas finales
    print(f"\n{'='*70}")
    print("ESTADÍSTICAS DE LA SIMULACIÓN:")
    print(f"  Total vendido: {total_vendido:,} unidades")
    print(f"  Ventas perdidas: {ventas_perdidas:,} unidades")
    print(f"  Nivel de servicio: {(total_vendido/(total_vendido+ventas_perdidas)*100)
    print(f"  Días sin stock: {dias_sin_stock} de {dias_simulacion}")
    print(f"  Número de pedidos: {num_pedidos}")
    print(f"  Inventario final: {inventario} unidades")
    print(f"  Inventario promedio: {inventario_inicial/2:.0f} unidades (aprox.)")

# Ejemplo de uso
simular_politica_inventario(
    demanda_diaria=50,
    dias_simulacion=90,
    punto_reorden=200,
    cantidad_pedido=500,
    tiempo_entrega=5,
    inventario_inicial=600
)
```

12 Conlusión

En esta guía hemos explorado las iteraciones en Python:

Actualización de variables

Uso de operadores compuestos para modificar variables: `+=`, `-=`, `*=`, `/=`.

Bucles for

Iteraciones definidas sobre secuencias. La función `range()` genera secuencias numéricas. `enumerate()` proporciona índices. `zip()` permite iteración paralela.

List comprehension

Forma concisa y eficiente de crear listas. Puede incluir condicionales y expresiones complejas.

Iteraciones anidadas
Permiten trabajar con estructuras multidimensionales como matrices y tablas.
Bucles while
Iteraciones indefinidas basadas en condiciones. Útiles cuando no sabemos cuántas iteraciones necesitamos.
Control de flujo
`break` termina el bucle completamente. `continue` salta a la siguiente iteración.

13 Próximos pasos

En la siguiente guía (Guía 6: Funciones) exploraremos:

- Flujo de ejecución, argumentos y parámetros
- Definición y uso de funciones
- Funciones anónimas (lambda)
- Funciones map y filter
- Módulos NumPy y Pandas

Las funciones nos permitirán organizar nuestro código de manera modular y reutilizable, fundamental para proyectos económicos complejos.

14 Recursos adicionales

Para profundizar en iteraciones:

- Python Loops: docs.python.org/tutorial/controlflow.html
- List Comprehensions: docs.python.org/tutorial/datastructures.html
- Práctica con datasets económicos reales

La práctica constante con datos reales consolidará estos conceptos. Se recomienda aplicar iteraciones en el procesamiento de datos del BCRP, INEI y otras fuentes económicas, automatizando análisis que normalmente requerirían horas de trabajo manual.

15 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1. Instalacion De Anaconda
2. Configurar Entorno Virtual Python Anaconda
3. 01 Introducion A La Programacion Con Python
4. 02 Variables Expresiones Y Statements Con Python
5. 03 Objetos De Python
6. 04 Ejecucion Condicional Con Python
7. 05 Iteraciones Con Python
8. 06 Funciones Con Python
9. 07 Dataframes Con Python
10. 08 Prediccion Y Metrica De Performance Con Python
11. 09 Metodos De Machine Learning Para Clasificacion Con Python
12. 10 Metodos De Machine Learning Para Regresion Con Python
13. Visualizacion De Datos Con Python

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!